

Strategy DP

22 May 2026 02:20 PM

Design Patterns are used to solve common software problems in a reusable and clean way.
The Strategy Design Pattern helps us avoid complicated inheritance and repeated code.
It follows an important principle:

Composition is better than inheritance.

Problem with Inheritance

Suppose we have a Robot class with methods like:

```
walk()
talk()
projection()
fly()
```

Different robots inherit from this class.

Example:

- SparrowRobot
- CrowRobot
- CompanionRobot

Some behaviors are common:

- walking
- talking

But some behaviors are different:

- flying
- appearance/projection

The Main Issue

Suppose both SparrowRobot and CrowRobot have the same fly() method.

Then we are writing the same code again and again.

This violates the DRY principle:

DRY = Don't Repeat Yourself

First Attempt

We may create another class/interface called:

Flyable

and move flying behavior there.

Now all flying robots can reuse the same fly code.

This improves the design.

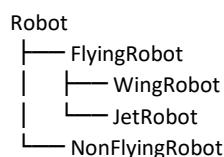
New Problem Appears

Later, we may have:

- robots flying with wings
- robots flying with jets
- robots that cannot fly

Again inheritance becomes complicated.

Example:



Hierarchy keeps growing and becomes difficult to manage.

Better Solution → Strategy Pattern

Instead of inheritance, separate the changing behaviors into interfaces/classes.

Behaviors that can change:

- walk
- talk
- fly

Keep them independent.

Create Separate Interfaces

Walk Behavior

```
interface Walkable {  
    void walk();  
}  
Implementations:  
• NormalWalk  
• NoWalk
```

Talk Behavior

```
interface Talkable {  
    void talk();  
}  
Implementations:  
• NormalTalk  
• NoTalk
```

Fly Behavior

```
interface Flyable {  
    void fly();  
}  
Implementations:  
• NormalFly  
• JetFly  
• NoFly
```

Robot Class Uses Composition

Now the Robot class will contain objects of these behaviors instead of inheriting everything.

```
class Robot {  
  
    Walkable walkable;  
    Talkable talkable;  
    Flyable flyable;  
  
    Robot(Walkable w, Talkable t, Flyable f) {  
        this.walkable = w;  
        this.talkable = t;  
        this.flyable = f;  
    }  
  
    void walk() {  
        walkable.walk();  
    }  
  
    void talk() {  
        talkable.talk();  
    }  
  
    void fly() {  
        flyable.fly();  
    }  
}
```

The Robot class delegates the work to behavior classes.

Example

Companion Robot

A companion robot:

- can walk
- can talk
- cannot fly

```
Robot robot = new Robot(  
    new NormalWalk(),
```

```

        new NormalTalk(),
        new NoFly()
    );

```

Advantage

Now if we want a new flying style like:

JetFly

we only add a new class.

No need to modify existing robot classes.

This follows:

- Open Closed Principle
- Code Reusability
- Loose Coupling

Real-World Example

Payment applications use Strategy Pattern.

Different payment methods:

- UPI
- Credit Card
- Debit Card
- PayPal

All follow the same payment interface:

pay()

But implementation changes.

So payment method can be changed at runtime easily.

Definition of Strategy Design Pattern

Strategy Pattern defines a family of algorithms, puts them into separate classes, and makes them interchangeable at runtime.

using System;

```

// ----- Strategy Interface for Walk -----
interface IWalkableRobot
{
    void Walk();
}

```

```

// ----- Concrete Strategies for Walk -----
class NormalWalk : IWalkableRobot
{
    public void Walk()
    {
        Console.WriteLine("Walking normally...");
    }
}

```

```

class NoWalk : IWalkableRobot
{
    public void Walk()
    {
        Console.WriteLine("Cannot walk.");
    }
}

```

```

// ----- Strategy Interface for Talk -----
interface ITalkableRobot
{
    void Talk();
}

```

```

// ----- Concrete Strategies for Talk -----
class NormalTalk : ITalkableRobot
{
    public void Talk()
    {
        Console.WriteLine("Talking normally...");
    }
}

```

```

    }
}

class NoTalk : ITalkableRobot
{
    public void Talk()
    {
        Console.WriteLine("Cannot talk.");
    }
}

// ----- Strategy Interface for Fly -----
interface IFlyableRobot
{
    void Fly();
}

// ----- Concrete Strategies for Fly -----
class NormalFly : IFlyableRobot
{
    public void Fly()
    {
        Console.WriteLine("Flying normally...");
    }
}

class NoFly : IFlyableRobot
{
    public void Fly()
    {
        Console.WriteLine("Cannot fly.");
    }
}

// ----- Base Robot Class -----
abstract class Robot
{
    protected IWalkableRobot walkBehavior;
    protected ITalkableRobot talkBehavior;
    protected IFlyableRobot flyBehavior;

    // Constructor Injection
    public Robot(
        IWalkableRobot walk,
        ITalkableRobot talk,
        IFlyableRobot fly)
    {
        walkBehavior = walk;
        talkBehavior = talk;
        flyBehavior = fly;
    }

    // Delegating behavior
    public void Walk()
    {
        walkBehavior.Walk();
    }

    public void Talk()
    {
        talkBehavior.Talk();
    }

    public void Fly()
    {
        flyBehavior.Fly();
    }

    // Abstract method
    public abstract void Projection();
}

```

```

// ----- Concrete Robot Types -----
class CompanionRobot : Robot
{
    public CompanionRobot(
        IWalkableRobot walk,
        ITalkableRobot talk,
        IFlyableRobot fly)
        : base(walk, talk, fly)
    {
    }

    public override void Projection()
    {
        Console.WriteLine("Displaying friendly companion features...");
    }
}

class WorkerRobot : Robot
{
    public WorkerRobot(
        IWalkableRobot walk,
        ITalkableRobot talk,
        IFlyableRobot fly)
        : base(walk, talk, fly)
    {
    }

    public override void Projection()
    {
        Console.WriteLine("Displaying worker efficiency stats...");
    }
}

// ----- Main Program -----
class Program
{
    static void Main(string[] args)
    {
        // Companion Robot
        Robot robot1 = new CompanionRobot(
            new NormalWalk(),
            new NormalTalk(),
            new NoFly()
        );

        robot1.Walk();
        robot1.Talk();
        robot1.Fly();
        robot1.Projection();

        Console.WriteLine("-----");

        // Worker Robot
        Robot robot2 = new WorkerRobot(
            new NoWalk(),
            new NoTalk(),
            new NormalFly()
        );

        robot2.Walk();
        robot2.Talk();
        robot2.Fly();
        robot2.Projection();
    }
}

```